Create a run

Boxes are types/data. Arrows are functions. A dot joins required inputs; repeated names refer to the same object.

01 LOAD THE SCENARIO

dataset + date

str + str

e.g. "i24", "11_30"

`load_scenario(dataset, date)`

reads observation files + dataset metadata

Scenario

spec: ScenarioConfig - identity, L, segment count, time step, duration

traffic: TrafficData - density, velocity, flow, lanes, boundaries and initial density / velocity

02 OBTAIN PARAMETERS: LOAD SAVED VALUES OR FIT NEW ONES

Scenario

the scenario loaded above

CalRef

source: CalSource | str

interval: int | None

`load_params(...)`

reads saved parameter files

CalibrationConfig

ramps, tolerances, starting guesses

robust_opt: RobustOptConfig | None

None = nominal; otherwise robust fitting

`calibrate(...)`

MetanetParams

tau, K, eta_high, p_crit, v_free, a

q_capacity, r, beta, gamma

TypedDict of spatial or time-space arrays

Simulation-ready parameter values

Both routes return this same type.

calibrate(scenario, config) fits values;
it does not automatically save calibration files.

03 BUILD THE MPC SETTINGS

ScenarioConfig

scenario.spec

resolved geometry + timing

`.to_mpc_config(**overrides)`

MPCConfig

horizons, hold length, speed bounds, control zone, safety limits

initialization: a speed, InitMode.ADAPTIVE, fallbacks, or None

04 OPTIMIZE

Scenario

MetanetParams

MPCConfig

solves MPC, then evaluates the policy

`optimize(...)`

RunResult

scenario + params + config

optimization: OptimizationResult

Save, reload and report

CalRef and Study choose storage locations. They are not additional numerical inputs to optimize().

05 SAVE A COMPLETE RUN

RunResult
full inputs + settings + outputs
from optimize()

CalRef
source + optional interval
directory identity

StudyChoice
None, SPEED_LB, HOLD_LENGTH,
or SAFETY_SWEEP

Study is a folder label. Set actual constraints on MPCConfig.

`save_result(...)`

Path to .npz
serialized RunResult
default filename: "run.npz"
existing target is replaced

RESULT DIRECTORY

```
results/{dataset}/{dataset}_{date}/{calibration.source}/
  [control_h_{interval}/][{study}/]{file_name}.npz
```

collection_dir(...) builds the folder; find_npzs(...) lists matching bundles.

06 RESTORE A RUN: BY ITS FILE PATH OR BY LOOKUP

`load_npz(path)`

dataset + date + CalRef + StudyChoice
the lookup key; study defaults to None
optional where: filter MPCConfig values
load_results also accepts a filename pattern (default "*")

`load_one_result(...)`

RunResult
restored from saved arrays
no re-fitting or optimization

`load_results(...)`

list[RunResult]

Loading restores the snapshot; it does not read current observation or calibration files.

07 REPLAY THE POLICY, THEN COMPUTE REPORTS

`replay(run, until_ff=True)`

RunResult
same inputs, config and policy
fresh simulation histories

`run_analysis(run)`

MPCStats
TTT, free-flow TTT, delay, CC,
fit errors, per-vehicle savings

Plot inputs
RunResult + MPCStats + pct_decrease + output_dir
pct_decrease is prepared by experiments/cc_plot.py

`save_cc_plots(...)`

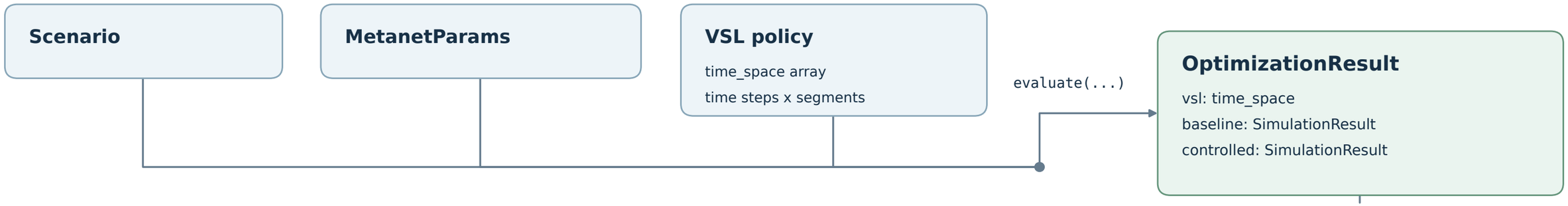
PNG figures

save_reveal_plots(run, output_dir) also produces progressive-reveal PNGs.
It re-simulates partial policies; it does not solve MPC.
Table, constraint and TSD scripts use replay(...) + run_analysis(...).

Evaluation and result types

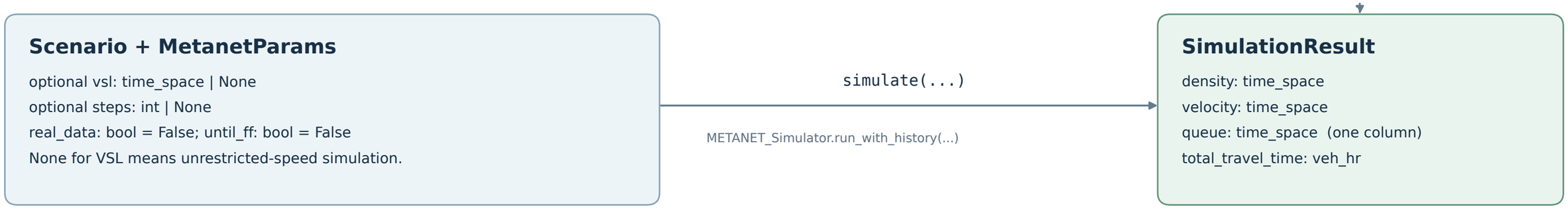
Solid arrows transform data. Dashed arrows identify fields contained in a result. No MPC solve is needed for evaluation.

A EVALUATE A SUPPLIED POLICY



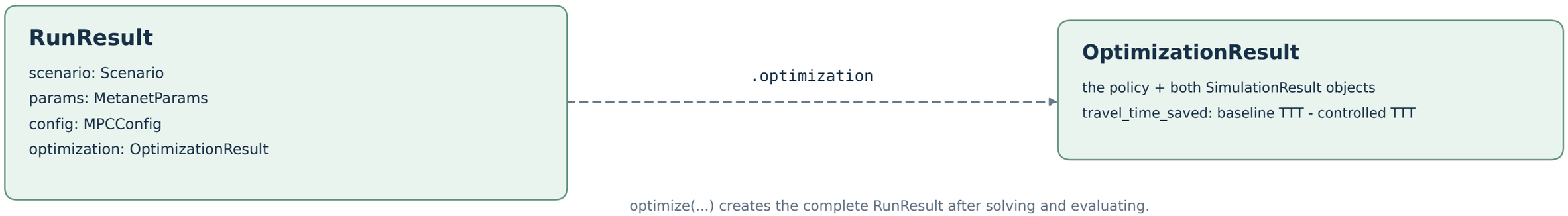
evaluate(scenario, params, vsl, until_ff=False) calls simulate(...) twice:
baseline: no VSL, measured inflow | controlled: supplied VSL, modeled origin / queue.

B SIMULATE ONE TRAJECTORY



Histories include the terminal state. With drainage, the two histories can have different lengths.
The supplied VSL matrix still covers the original horizon; drainage releases the speed limits.

C HOW THE COMPLETE RESULT CONTAINS THESE OUTPUTS



optimize(...) creates the complete RunResult after solving and evaluating.
replay(...) replaces only its optimization field with a fresh evaluate(...) result.